THE STANDARD HARNESS

How to build an agent on this framework. The rest of the folder is evidence; this file is the recipe.

PROVEN, OR MARKED DESIGN. NEVER ASSERTED.

```
trigger → preflight → workflow → agent → durable state → memory write → freshness check → grade
```

Everything here is either **proven** by the tests in `HARNESS.md` / `MEMORY.md` / `INNGEST.md`, or marked as **design** where it isn't. That distinction is the point — see How to read it.

SECTION 01 · ARCHITECTURE

1. THE SHAPE

```
TRIGGER                    whatever the OS dispatches
   │                       • a fellow request via Agent Inventory  ← the primary path
   │                       • a signal or webhook                    ← event-driven
   │                       • a schedule                            ← recurring
   ▼
1. PREFLIGHT               can I reach my dependencies?
   │                       no → record a clean `offline`, exit 0, don't start work
   ▼
2. WORKFLOW                Mastra workflow — the orchestration. Deterministic.
   │                       steps are named and typed; sub-modules are nested workflows
   │                       ├─ suspend() ──► human approval ──► resume()
   ▼
3. AGENT, inside a step    Agent.generate() — model judgement, scoped to one decision
   │                       skills are its tools
   ▼
4. DURABLE STATE           ConvexStore + ConvexVector
   │                       workflow snapshots · memory · vectors · runs
   ▼
5. MEMORY WRITE            deterministic, after the step. NOT the model's choice.
   ▼
6. FRESHNESS CHECK         did the write actually land? check the timestamp, never the size
   ▼
7. GRADE                   BEHAVIOR.md against the trace, afterwards, out of band
```

On the trigger: we tested with `launchd` because a recurring schedule is the *harshest* durability test available — it runs unattended for days across laptop sleep, lid closes and network loss, with nobody watching. That was the test rig, **not the recommendation.** In production the OS dispatches: a fellow brings a request to Agent Inventory, or a signal fires. The seven steps below are identical either way; only what calls step 1 changes.

Steps 1, 5 and 6 exist only because they broke. They are not obvious, and none of them are in Mastra's docs. Each one cost a day.

> **Plain-English version:** a trigger starts a run; code checks whether it is safe to begin; a workflow owns the known sequence; an agent makes the one judgement that needs a model; and the system writes down enough evidence to resume, explain, and grade the result later.

1. Start safely	2. Do the work	3. Keep evidence	4. Improve over time
trigger + preflight	workflow + agent	Convex + event log	memory + grading

---

1a. How a request reaches the harness — the primary path

Step 1 above starts with a trigger, and for a fellow-initiated run that trigger has a shape worth specifying, because a workflow takes **typed input** and a fellow types prose.

```
FELLOW                    Slack, or the Agent Inventory UI
   │                      free text: "20 qualified leads for fintech CFOs, outreach drafted"
   ▼
ROUTER                    one model call, and its only job
   │   IN   free text
   │   OUT  { agentId, typed input }        ← validated against that agent's schema
   │
   ├── can't fill it confidently ──► asks the fellow. NOTHING STARTS.
   ▼
CONVEX · RUN ROW          runId · tenant · fellow · agent · input · status=queued
   │                      written BEFORE dispatch
   ▼
MASTER WORKFLOW           the seven steps above
```

The router picks the *agent*, and the module follows from it. "Which module" is not enough to run anything — a module holds several agents, and the typed input belongs to the agent, not the module. So the router has to resolve to agent level, which is also the only level at which the form can be filled.

Three rules, and the first is the one that saves money:

- **Nothing starts until validation passes.** A half-understood request is a *question back to the fellow*, not a guess. An agent started on a bad payload burns budget and returns something plausible and wrong —

which is worse than an error, because nobody catches it.

- **The run row is written before dispatch**, so a request that never started is still visible. Generalises `HORIZON-4`: a precondition that leaves no trace is not auditable.
- **The router picks and stops.** It never orchestrates. One judgement, handed off — see the division of labour in §2.

The return leg, which is where designs usually break

An approval can arrive days later, in a different process, on a different device.

```
suspend()  ──►  run row: status=awaiting_human, suspension path stored
                        │
                        ▼
                NOTIFY the fellow, on the surface they came in on
                the message carries the runId and NO state
                        │
                        ▼
                fellow approves  ──►  resume({ step, resumeData })
                                      loads the run from Convex by runId
                                      IDEMPOTENT — approving twice is one approval
```

Idempotency here is not hygiene. It is the difference between *"we sent the outreach"* and *"we sent the outreach twice"* — `LOOP-6`.

What exists, and the one thing that blocks this

The registry exists. `schemas/agent-manifest.schema.json` requires `agent.id`, `agent.job` (one line, which is what a router picks on), `agent.module`, and `agent.non_goals` — so an agent can be chosen from the manifest rather than from a string the model invented. `lifecycle.tool_identity` is also required, which forces the *whose identity do the tools act as* question per agent instead of leaving it to discovery.

What is missing is a typed input schema per agent. The manifest describes an agent well enough to *pick*, but carries nothing to *validate a filled payload against*. Until it does, the router's `OUT` cannot be checked, and "nothing starts until validation passes" has nothing to check against. That is the single blocking gap on this path.

Scaffold (checkable, not the full path): `intake/` validates router `OUT { agentId, input }` against a per-agent schema and refuses half-filled or unknown-agent payloads (`node --test long-horizon/intake/tests/validate.test.js`). Envelope schema: `schemas/router-output.schema.json`. The router, run row, and notify/approve leg remain unbuilt.

Also unbuilt: the router itself, the run row, and the notify/approve leg. What is proven is the middle — a workflow suspending, surviving a hard kill, and resuming from Convex with the runId as its only input.

And one case with no answer: a request that spans modules. *"Is this segment worth building for"* is Product discovery **and** GTM validation. The router picks the closest single agent, and should say plainly what it is not covering rather than quietly doing half the job.

SECTION 02 · ORCHESTRATION

2. ORCHESTRATION — A MASTRA WORKFLOW, WHERE ONE IS WARRANTED

First: does this agent need a workflow at all?

This is a per-agent decision, not a default, and it belongs to the builder workflow — see §2a. The question is not "is this agent important". It is:

> **Does losing work mid-flight cost anything?**

WHAT THE AGENT IS	WORKFLOW?	WHY
One model call, no state	**no**	nothing exists to resume
Independent cycles, cheap to redo	**no**	a lost cycle costs one interval
A human has to approve something	**yes**	`suspend()` / `resume()` *is* the gate, and it must survive days
State accumulates across steps	**yes**	mid-flight loss is expensive
Sub-modules durable on their own	**yes**	a nested workflow gets its own snapshot
Conduct graded step by step	**yes**	named steps are a trace; a loop is a transcript

Row two is not a loophole — it is our own reference agent. It ran 46 cycles over 41 hours through three sleep/wake boundaries with **no workflow at all**, just `Agent.generate()` on a trigger, and survived because its cycles are independent: a lost cycle costs one interval and the next tick catches up. A workflow would have bought nothing and cost a snapshot write per step.

A workflow "because it's the standard" with no answer to that question should be rejected in review. So should the reverse — no workflow where a human has to approve something.

2a. Who makes that call, and where it is recorded

The skill chain decides this before code is written, and carries the decision forward:

```
agent-builder      the front door — intake, then chains the stages
  ├─ workflow-design    stage 1 — fleet or solo, spawn triggers, loop exits
  ├─ agent-design       stage 2 — role · tools · memory layer · eval pointer
  ├─ eval-first-spec    stage 3 — 20 golden cases, autonomy L0–L4, cost per outcome
  ├─ agent-prd          stage 4 — hard gates → PRD → work orders → STOP
  └─ mastra-harness     stage 5 — workflow? memory? → implement → doctor
                                  ▲
                        this file is stage 5's reference
```

Stage 5 (`mastra-harness/`) exists because stage 4 ends with *"Stop. Do not begin executing the first order unless asked"* and nothing picked it up. It makes both per-agent calls — workflow yes/no, and which memory channels — records them in its `template.md` **with reasons**, and ships the runtime pieces as code. An empty cell in that record fails the gate: *"we'll decide later"* becomes *"we defaulted"*, and both defaults are wrong for some agents.

Why a workflow, when one is warranted

Durability is proven at exactly this layer. The STATE-1 kill-test — `PASS 12 · FAIL 0 · BLOCKED 0`, live model, real Convex — is *workflow snapshots resuming from Convex after a* `kill -9`. That is the verified thing in this whole folder. A loop where the model decides what to do next has no snapshot to resume from; when the process dies, the run is gone.

Named steps give you a gradeable trace. `BEHAVIOR.md` grades conduct — what happened, in what order, with what tools. Steps have ids, so the trace is structured data. A prompt-driven loop leaves a transcript you have to infer intent from, and inference is exactly what you can't rely on when you're checking whether the agent misbehaved.

Human approval is `suspend()` / `resume()`, and you get it for free. The Fellow → request → work → *"delivers output back"* round trip is literally a suspended workflow. The snapshot persists in Convex; the run resumes when the approval arrives, in a different process, possibly days later. Proven, including across a hard kill.

Failure localises. A typed step that fails names itself and its inputs. A failing agent loop just stops producing output, which is the hardest class of bug to find — and precisely the class this project kept hitting.

The division of labour

> **Deterministic orchestration. Model judgement inside steps.**
>
> If the control flow is knowable, don't let the model decide it.

The Master Agent for a harness is a **workflow**, not an agent. The agents live *inside* its steps, each scoped to one decision, with skills as their tools.

A workflow does not "decide" anything, and the phrasing matters. Saying *"the workflow decides which sub-module"* invites the obvious question — how? — and the honest answer is the whole architecture:

> The workflow runs a **step** that asks the model, and **the model's answer is written down as a value.** Code then acts on that value.

That is not a pedantic distinction. It is the difference between:

	WHAT THE STATE IS	KILL THE PROCESS
a step returning a value	a typed row in Convex	resume reads the value back. **Same answer, guaranteed**
a model reasoning in a loop	the conversation	nothing to resume from. It may decide **differently** next time

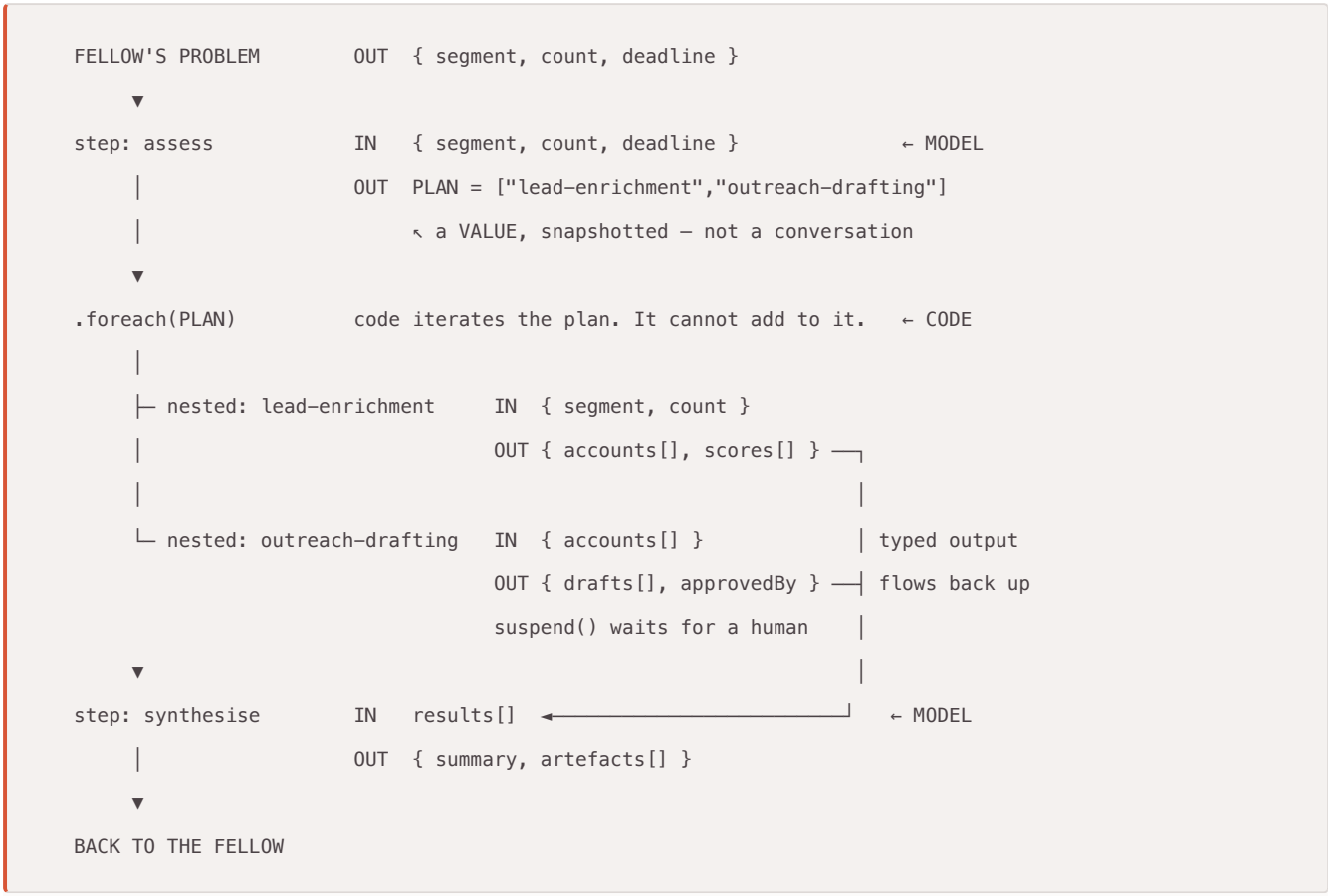
And deciding differently is not merely untidy — it re-runs the paid lookup, or sends the outreach again, because "*I already did that*" was in the conversation that died.

The primitives worth knowing

`createStep({ id, inputSchema, outputSchema, execute })`	a typed unit of work
`createWorkflow(...).then(step).commit()`	sequential composition
`.parallel([...])`	concurrent steps
`.branch([[condition, step], ...])`	conditional routing
`.dowhile()` · `.dountil()` · `.foreach()`	loops with a bound
`.map()`	reshape data between steps, instead of steps knowing each other's shapes
a workflow as a step	nesting — this is how sub-modules compose
`suspend()` / `resume()`	human-in-the-loop, snapshot-backed
`watch()`	observe progress without changing behaviour

Type every step, and the run becomes resumable for free

The reason a kill at any point is survivable is not the framework being clever. It is that each step declares what it takes and what it returns, so Convex has something to write down.



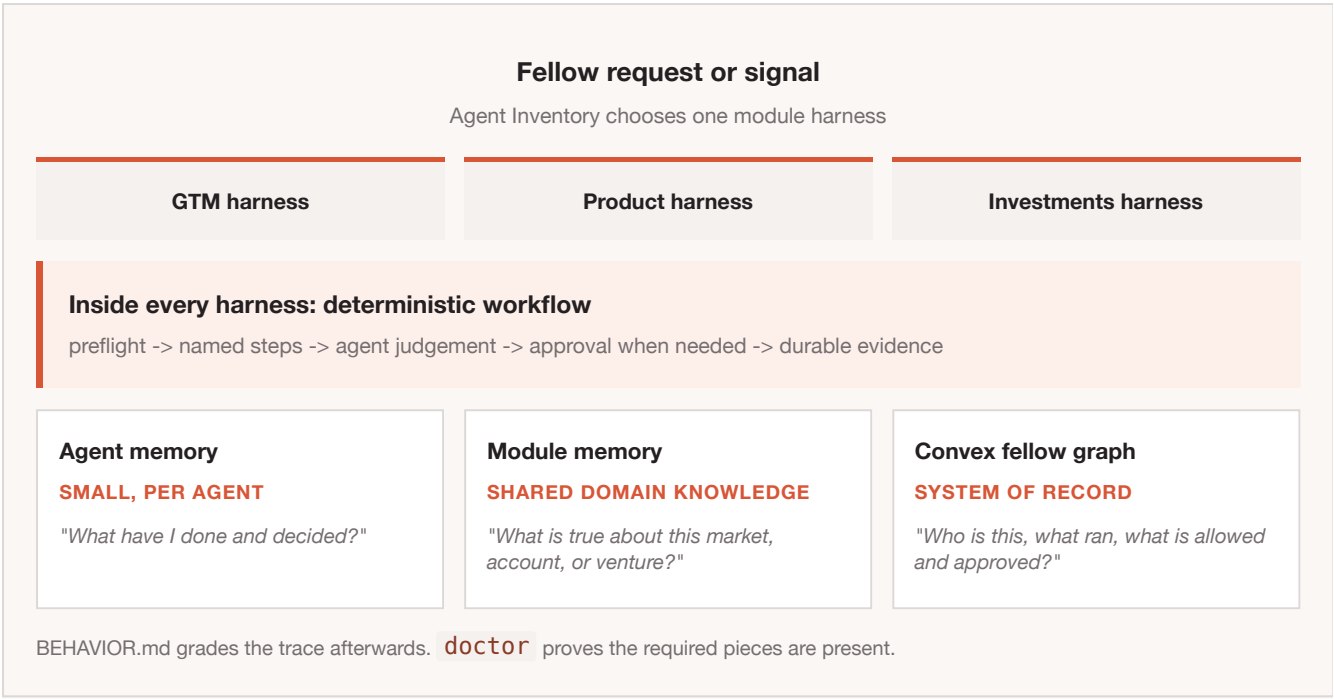
Every step's `IN` and `OUT` is written to Convex as the step completes. Kill the process at any arrow and resume reads the last recorded `OUT` and carries on with the same values. The model appears twice here and both times produces a *value* — a plan, or a synthesis. It never holds the route.

Note what the model **cannot** do in this shape: it cannot name a sub-module that isn't registered, and it cannot change its mind except at a step someone deliberately placed. Re-planning is a **location in the code**, not a setting — so per module, decide where the agent is allowed to reconsider, and record it. For GTM that is probably after enrichment returns and before drafting; almost certainly not mid-draft.

Status: untested by us. `.foreach()` over a model-produced plan, nested workflows returning typed output upward, and a suspend surviving a kill in the middle — each primitive exists and is documented, and we have run **none of that combination**. What is proven is the simpler shape: `.branch()` into one nested workflow with a `suspend()` inside it, hard-killed and resumed. Worth an hour's test before three modules depend on it.

How the modules, the workflow, the memory and the harness fit together

One picture, because these four are usually drawn separately and the seams are where things go wrong.



Read the bottom row as the load-bearing part. The three boxes sit side by side as an **ownership map**, not as three databases that all remember the same thing. Each has one question, and the rule that keeps them apart:

> **One question, one owner.** If two layers can answer the same question, the agent stops maintaining whichever one costs it effort.

That is not a design preference. It is the failure in `MEMORY.md` stated as a rule — two channels answered *"what have I already covered?"*, so the agent stopped writing the one that required a tool call. A third layer answering the same question reproduces it, larger.

Two structural notes on the interior:

- **Nesting is what makes a sub-module independently testable, gradeable and resumable.** It gets its own snapshot row, so it can be run, killed and resumed on its own — verified.
- **Not every module needs every piece.** Preflight and `doctor` are always there. The workflow itself, the memory write and the freshness check are conditional on the two per-agent decisions in §2 and §3. An agent with no memory must not be failed for having none.

Verified vs not

Verified here: workflow snapshots persist in `ConvexStore` and resume after `kill -9` (12/12 golden cases) · `suspend()` / `resume()` on a tool-approval gate.

And the nesting this section recommends is now tested too (2 Sep) — the exact shape above: a parent workflow, `.branch()` routing on real data into a **nested** workflow, `suspend()` inside that nested workflow, the process `kill -9`'d while suspended, and a fresh process resuming from Convex with the runId as its only input.

```
[nest] suspended at: [["submodule-approval","await-approval"]]
   kill -9 → process is gone
[nest] FRESH PROCESS resuming runId=24a2b4b0-…
[nest] status=success
[nest] result={"submodule-approval":{"verdict":"approved by haniyah"}}
```

Two structural details worth knowing before you build on it:

- **A nested workflow gets its own snapshot row — on the plain engine.** One run produced two rows sharing a runId: `module-harness` (the parent) and `submodule-approval` (the child). Sub-modules are durable independently, which is what makes them independently resumable.

  This does not hold on Inngest. The same shape run through `init(inngest)` writes **one** row, the parent only (verified 4 Sep). So "sub-modules are independently resumable" is an engine-specific property, not a property of nesting. If a design depends on it, it depends on the plain engine. Also on Inngest, `res.suspended` comes back `null` rather than the path — you must know the suspension path yourself. See `INNGEST.md`.

- **Suspension is addressed by path, two levels deep.** The parent records `suspendedPaths: {"submodule-approval": [1,0]}`; the child records `{"await-approval": [1]}`. `resume()` takes `step: ["submodule-approval", "await-approval"]` — the nested workflow id, then the step id. Build your approval routing around that shape.

The real table name, since we got this wrong once: `TABLE_WORKFLOW_SNAPSHOT = "mastra_workflow_snapshot"` — **singular**. The package's bundled reference doc says plural; that is an upstream documentation bug, and following it makes you declare an empty table while the one holding your data stays undeclared. Reads and writes are both singular and self-consistent, so nothing breaks — but declare the singular name so the real table gets a validator and indexes.

Still unverified by us: `.foreach()` · `.parallel()` · `.dowhile()` / `.dountil()` · `.agent()` as a step · `watch()` streaming · nesting deeper than two levels. The primitives exist and are documented upstream; we have not run them, and this folder does not claim otherwise.

And the entry path in §1a is mostly design, not evidence. The router, the run row, and the notify/approve leg are unbuilt. A checkable typed-input scaffold now lives in `intake/` (router `OUT` + per-agent schema + refuse-to-start tests); the manifest still does not embed that schema. So the *middle* of the picture is proven, the *input gate* has a small executable stub, and both *ends* of the front door remain proposed — worth saying before anyone demos it as a working entry path.

SECTION 03 · MEMORY

3. MEMORY — THREE RULES, ALL LEARNED THE HARD WAY

1. **Never write Mastra's memory tables directly.** Reading them raw is essential — it's the only honest verification. *Writing* them silently removes the agent's ability to maintain its own memory: the update tool stops being offered on the request at all.
2. **Don't leave the memory write to the model.** It will stop doing it. Reproduced A/B: with semantic recall available, the model declines to call `updateWorkingMemory` because recall already answers "*what have I covered?*". **Memory maintenance decays as the corpus grows** — backwards for a long-horizon agent. Write it deterministically after the step.

   And you cannot bolt that on top of the feature. With `workingMemory: { enabled: true }`, Mastra appends "*IMPORTANT: You MUST call updateWorkingMemory in every response…*" to the system prompt **after** your instructions; the model gets contradictory orders and satisfies neither. Use **two** `Memory` **instances** — the agent's with `workingMemory: false`, plus a storage-only one never attached to an Agent for `get` / `update`. Then reading is your job too, which is an improvement: the memory sits in a prompt you wrote. Full account in `MEMORY.md`.
3. **Check the write timestamp, never the size.** A frozen memory and a healthy memory are the same number. Nine cycles reported `ok`, recall passed on all nine, memory read a plausible 1,742 chars — and it hadn't been written once.

> **Grade freshness, never size.** A metric you can satisfy by doing nothing is not a metric.

Full account in `MEMORY.md`.

SECTION 04 · RUNTIME

4. LONG-RUNNING BACKGROUND AGENTS

Everything above assumes a **request-shaped** run: a fellow asks, a workflow answers, the run ends. A background agent is a different shape — it runs for hours or days with nobody watching, and it has no single end. This section answers the six questions that shape leaves open. All six answers come from a **41-hour unattended run** across three sleep/wake boundaries, one of them with the laptop shut in a bag and no network.

4a. Trigger cadence and ownership

Schedule-driven is verified: two agents at ~20 min and ~30 min intervals, 71 runs and 46 cycles, two non-ok between them. Event-driven uses the same seven steps — only what calls step 1 changes. Always-on is **untested** and should not be assumed to work by analogy.

The trigger config lives **with the agent**, versioned alongside it, not in a central schedule nobody owns.

> **Assume the trigger is hostile.** It will fire at the worst possible moment — during a two-second macOS DarkWake with the network stack still initialising, mid-sleep, with no human awake. Every background tick must therefore be **idempotent** and able to **no-op cleanly**. An agent that can only succeed is an agent that will hang.

4b. Checkpointing across runs, not just within one

The sharpest distinction in this section, and the one most easily got wrong:

	CARRIES	SURVIVES
Workflow snapshot	state *within* one run	process death mid-run (`kill -9`)
Memory (`mastra_resources`)	state *across* runs	the run ending, normally or not
Domain store	the agent's actual output	everything

A workflow snapshot is not a checkpoint across runs. It exists to resume a run that was interrupted. If your agent's cycles are separate runs — which is what a schedule produces — then cross-run continuity is **memory**, plus whatever domain store holds the work product. Reaching for snapshots to carry day-to-day state is the wrong tool and will look like it works until the first clean run boundary.

Verified: cycle 2 in a **separate OS process** recalled what cycle 1 wrote, with no application code carrying anything between them, read back over raw HTTP with zero SDK.

And the caveat that matters more than the capability: cross-run memory **decays unless the write is deterministic**. Nine consecutive cycles reported `ok` while durable memory was never written — see §3. For a background agent this is the single most dangerous failure mode available, because there is no human in the loop to notice that the notes stopped updating.

One honest limit. A cycle that dies *before* it has state worth resuming is **abandoned, not resumed** — nothing retries it, and the next scheduled tick starts fresh. That is weaker than the kill-test and must not be conflated with it:

- **Kill-test** — a suspended run *with real state in Convex* resumes. Proven.
- **Cold-network cycle** — died before it had state. Abandoned. No retry exists today.

For an agent whose cycles are independent this costs one interval. For one with a real task in flight it would matter, and there is currently no answer.

4c. Timeout, retry, and knowing it didn't run

Four statuses, not two. `ok` · `degraded` · `offline` · `failed`. The distinction earns its keep: `offline` means the *environment* failed and there is nothing to debug in the agent, while `failed` means the agent did. Collapsing them produces nightly false alarms that get ignored, which is worse than no alarm.

Timeout is a preflight problem, not a timeout problem. The worst observed failure was a **44.6-minute hang** — a DarkWake fired the job with no network, the model call died *inside* the workflow, so the cycle recorded as `failed` and Mastra could not clean up its own snapshot rows, leaving orphaned `pending` state. Over a week of DarkWakes that debris accumulates and every row reads as a genuine fault. Probe dependencies *before* starting work and a network-shaped error becomes a 3.6-second `offline` skip. Also set a hard timeout on the model call; the preflight narrows the window, it does not close it.

"The agent didn't run" means an *unexplained* gap — not a gap. Cross-check every gap against the machine's own sleep log and classify it:

VERDICT	MEANING
`asleep`	the machine was shut. **Not a failure.**
`jitter`	scheduler drift within tolerance
`partly-unexplained`	some of the gap is accounted for
`unexplained`	**this is the one that pages someone**

And **measure coverage against awake time, not wall-clock.** Judging a laptop on wall-clock scores it badly for being closed, which is not a fault — an early version scored a healthy agent at 18% that way.

A run can be `ok` and still not be doing its job, which is why status alone is not miss-detection. Pair it with a freshness check on whatever the agent is supposed to be maintaining (§3). Today the alarm surface is the dashboard plus `doctor` exiting non-zero; **there is no paging and no retry.** Say so rather than implying there is.

4d. Cost ceiling

An unattended agent needs a **hard** ceiling, not a warning. A soft warning stops nothing at 3am.

The guard that works: track spend per call in the agent's own ledger, and at the cap **unload its own scheduled job**. An agent that can't stop itself isn't capped. Verified: 41 hours, 125 model calls, 240,602

tokens, **$0.19 against a $3 cap** — 6.3%.

> **Budget from the late figure, never the empty one.** Memory makes input grow, so the first cycle's token count understates the steady state badly.

AGENT	AVG INPUT, FIRST 10	AVG INPUT, LAST 10	GROWTH
memoryless	421 tok	526 tok	+25%
with memory	1,043 tok	**1,678 tok**	**+61%**

Budgeting a memory-carrying agent from its first cycle underestimates by roughly 60%, and the curve had not plateaued when we stopped measuring.

4e. Where durability actually lives

Mastra workflows + `ConvexStore`. Verified to the nested case: parent → `.branch()` → nested workflow → `suspend()` inside it → `kill -9` → fresh process resumed from Convex with the runId as its only input.

`createInngestAgent()` **is out.** Inngest itself is sound — it holds a suspended run across a `kill -9`, re-invokes the worker unprompted, and Connect mode needs no tunnel and reconnected on its own after an 11-hour sleep. The resumed run then never completes. Five blockers, all in Mastra's durable-agent layer, in `INNGEST.md`. Revisit when they clear; the machinery underneath is the part that would be hard to build.

4f. Grading a run that has no end

`BEHAVIOR.md` assumes a trace with a start and a finish. A background agent's trace has neither. The resolution is to change the unit:

> **The unit of grading is the cycle, not the run.** Grade mechanical predicates over a **window of N cycles** rather than over a run boundary that never arrives.

Concretely, the predicate that would have caught the nine-cycle failure on cycle one:

> `required: updateWorkingMemory` — offered on the provider request **and** called, at least once per N cycles. Graded from the request/response, never from the resulting state.

This is why background agents make the case for conduct grading more sharply than request-shaped ones do. A request-shaped run has a human at the end who notices a bad answer. A background agent has nobody, so **the only thing standing between "working" and "quietly stopped" is a predicate over its behaviour** — and no outcome-based check can substitute, because a frozen state and a healthy state are the same value.

What a background agent ships that a request-shaped one doesn't

PIECE	BECAUSE
dependency preflight	the trigger fires with no network, and the failure is a hang
four honest statuses	`offline` is the environment; `failed` is the agent
gap attribution vs. the sleep log	a gap is not a miss until it is unexplained
coverage against **awake** time	wall-clock punishes a machine for being closed
freshness check on durable state	a cycle can be `ok` and still not be doing its job
a spend guard that **stops itself**	nobody is awake to read a warning
idempotent, no-op-able ticks	the trigger will fire at the worst moment

Still open

- **No retry for an abandoned cycle** (§4b). Independent cycles make this cheap; a real task in flight would not.
- **No paging.** The alarm surface is a dashboard and a non-zero exit.
- **Always-on agents** — untested. Only schedule- and event-driven shapes have been run.
- **A single run spanning days.** Ours are many short cycles, not one long run; `untilIdle` and the durable-agent APIs that would support that shape are unexercised.
- **Whether the cost curve plateaus.** It stepped up once and had not flattened.

SECTION 05 · GRADING

5. BEHAVIOR.MD — WHERE IT GOES, AND WHAT IT'S FOR

- It lives **next to the agent**, versioned with its code. **The agent never reads it.** It grades the trace afterwards, out of band. Injecting it turns a grading standard into a prompt.
- It grades **conduct** on a track *parallel* to output grading, not instead of it. Both green means ship.
- Start with the **mechanical** predicates, not the clever ones. "*Was the memory tool offered, and was it called?*" is two lines, needs no judgement, and would have caught nine hours of failure that four other signals missed.
- Predicates: `ordering` · `pairing` · `required` · `forbidden` · `count`, each returning true / false / `na`. The `na` state is load-bearing — without it every clause must be answerable on every run, which forces either false failures or clauses too weak to grade anything.

One clause we got wrong first, kept as a warning: "*memory must change each cycle*" would have failed a **correctly behaving** agent for seven consecutive cycles, because the input was genuinely already covered. Unchanged state on already-covered input is *correct*. A judged clause compares **input against state** — it doesn't just watch the state.

Blocked on one thing: we don't emit tool-*offering* in traces, only tool-calling. That's TUS-2758's scope, and the memory failure is the argument for adding it.

SECTION 06 · DURABILITY

6. INNGEST — NOT YET, AND BE PRECISE ABOUT WHY

Inngest itself works. It held a suspended run across a `kill -9`, kept state in Convex, and re-invoked the worker on its own with nobody driving it. Connect mode needs no tunnel or public URL, and reconnected unprompted after an 11-hour laptop sleep. Steps 4, 5 and 6 of the test are the hard parts, and all three pass.

Mastra's durable-agent wrapper on it does not. The resumed run never completes — `Cannot read properties of undefined (reading 'agentSpanData')`. Five blockers, all Mastra-side: a hardcoded workflow id that can't find its own run, and `createInngestAgent()` returning a plain object with none of the three recovery methods `createDurableAgent()` has.

So: Mastra workflows + `ConvexStore` is the standard today. `createInngestAgent()` stays out until the blockers clear — the machinery underneath is sound, and step 6 passing is the part that would be hard to build ourselves. Detail in `INNGEST.md`.

Also worth correcting, because it's in circulation: durable agents are **not** new in 1.62/1.63. `createInngestAgent` shipped in 1.30.0 and `untilIdle` in 1.41.0 — both already present in the 1.61.0 baseline the 26 Aug decision was made against.

SECTION 07 · MEMORY ARCHITECTURE

7. MEMORY ACROSS THE THREE MODULE HARNESSES

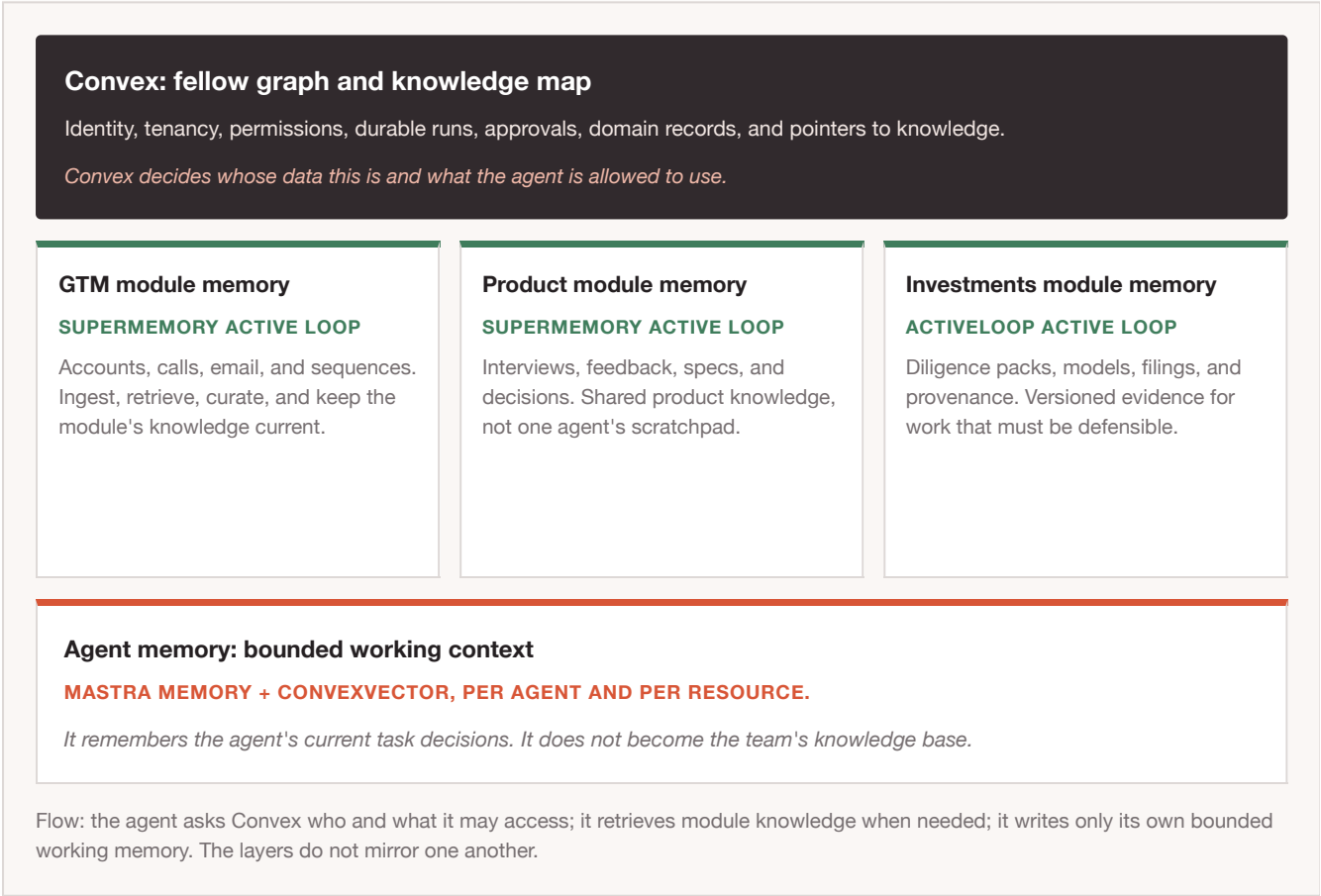
We offer three modules — **Product**, **GTM**, **Investments** — each with its own harness, a Master Agent orchestrating sub-modules, and agents with skills inside them. That structure raises a memory question the single-agent tests don't answer: *what remembers what, and at which layer?*

The boundary rule, and why it's not optional

The failure in `MEMORY.md` was **two memory systems answering the same question.** The agent stopped writing its own notes because another channel could answer instead. Add a third system that answers the same question and you get the same failure, larger and harder to see.

> **One question, one owner.** If two layers can answer the same question, the agent stops maintaining whichever one costs it effort.

Which gives three layers with three genuinely different jobs:



Flow: the agent asks Convex who and what it may access; it retrieves module knowledge when needed; it writes only its own bounded working memory. The layers do not mirror one another.

Read the three questions carefully, because the whole design is in them:

LAYER	ANSWERS	NEVER ANSWERS
Module memory	*what is true about the world?* — the account, the venture, the market	*have I already done this?*
Agent working memory	*what have I done and decided on this task?*	*what is true about the account?*
Convex fellow graph	*who is this, what ran, what is allowed, and which knowledge belongs to them?*	the full module corpus or an agent's scratchpad

Do not adopt a memory vendor to replace `ConvexVector` in the harness. Our vectors work — 83 of them, local 384-dim embedder, index created unprompted, retrieval verified. The argument for a module memory layer is *never* "better vectors". It is that the **module corpus outlives and spans individual agents**, and agent working memory is deliberately small and bounded.

Which vendor for which module

Neither is tested here. This is architecture, not a finding.

	SUPERMEMORY	ACTIVELOOP (DEEP LAKE)
What it is	a managed memory **service**	a versioned data **store** you run
Shape	ingest, and it decides retrieval	you own the retrieval logic
Strong at	fast to adopt, connectors, conversational memory	multi-modal, dataset versioning, self-hostable
Costs you	opinionation, an external dependency on the hot path	engineering time and ops

GTM → Supermemory. *"What do we know about this account?"* is a retrieval question. The corpus is conversational and arrives through connectors — Slack, email, Notion, call notes. Speed of adoption matters more than provenance.

Product → Supermemory. Interviews, feedback, specs, decisions. Same shape as GTM: retrieval over a growing conversational corpus.

Investments → Activeloop. Versioning is the deciding feature, not a bonus. *"Why did we believe that in Q2?"* is a **provenance** question, and diligence needs an auditable trail over multi-modal documents. A managed service that silently re-ranks is the wrong tool for a memo you may have to defend.

Sequencing: Supermemory first, on GTM or Product. Convex already holds identity and tenancy, so the memory layer starts as a queryable side-car you can remove. Activeloop's versioning becomes the stronger argument once *"why did the agent believe that last quarter?"* is a question someone actually asks — which, for Investments, it will be.

Wiring it without repeating the failure

1. **Give it one job, stated out loud.** *"Retrieve module knowledge"* — never *"remember things"*. Vague ownership is how you end up with two layers answering one question.

2. **Never on the hot path unguarded.** Same preflight rule as the model: unreachable memory service → degrade, don't hang. Our 44-minute stall came from exactly this shape of mistake.
3. **Tenancy at the boundary.** Every query carries a tenant scope, enforced where the identity lives — Convex — not inside the vendor.
4. **Grade it.** A `required: retrieveModuleMemory` predicate, **offered and called**. Managed services fail quietly and slowly, which is the exact failure mode nothing was watching.
5. **Verify by disabling everything else.** Turn the other channels off and confirm retrieval still works. Otherwise you are measuring one layer and crediting another — the mistake that cost this project two days.

Before adopting either — three tests

The same shape as the harness tests, because they're the ones that caught real problems:

- **Read the state back from outside the vendor's SDK.** If verification needs their client, you've proven a cache, not a store.
- **Kill the process mid-write.** Does it recover, or leave debris?
- **Run with harness memory disabled, then with the module layer disabled.** If behaviour is identical either way, the layers overlap — and one of them will stop being maintained.

SECTION 08 · REFERENCE

8. THE SHORT VERSION

DECISION	ANSWER
Harness	**Mastra + ConvexStore.** Proven: 12/12 kill-test, 41 hours unattended, three sleep boundaries
Orchestration	**Mastra workflows — where warranted.** A per-agent call (§2): a workflow only if losing work mid-flight costs something. Our reference agent used none and ran 41 hours
Who decides	**stage 5,** `mastra-harness`**.** Workflow yes/no and which memory channels, recorded with reasons. An empty cell fails the gate
How a fellow gets in	**a router picks the *agent*** (the module follows), fills its typed input, and stops. Nothing starts until validation passes; the run row is written before dispatch. **Typed-input scaffold in** `intake/`**; router/run-row still design** — §1a
What a "decision" is	a **step returns a value**, never a model reasoning in a loop. The value is snapshotted, so resume gets the same answer; a conversation is lost and may decide differently
Agents	inside workflow steps, one decision each, skills as tools
Sub-modules	**nested workflows** — independently runnable, independently gradeable
Human approval	`suspend()` / `resume()`, snapshot-backed
Agent memory	`@mastra/memory` + `ConvexVector`, **written deterministically**, freshness-checked. Fewest channels that answer the question — they compete
Module memory	Supermemory (GTM, Product) · Activeloop (Investments) — **untested, architecture only**
System of record	**Convex fellow graph.** Identity, tenancy, permissions, runs, approvals, and the knowledge map
Conduct grading	`BEHAVIOR.md`, out of band, next to the agent, never in context
Background agents	schedule- and event-driven **proven** (§4). Cross-run state is *memory*, not snapshots. Four honest statuses; gaps attributed against the sleep log; a spend guard that stops itself
Grading a run with no end	**the unit is the cycle, not the run** — mechanical predicates over a window of N cycles
Background durability	**not** `createInngestAgent()` **yet** — 5 blockers, all Mastra-side
Not answered	no retry for an abandoned cycle · no paging · always-on untested · a single multi-day run untested
Pin	`@mastra/convex` **1.5.4**. 1.5.5 fails the kill-test

Stack: `@mastra/core` 1.63.2 · `@mastra/convex` 1.5.4 (pinned) · `@mastra/memory` 1.27.0 · `@mastra/inngest` 1.8.8 · `@mastra/langfuse` 1.5.3 · `@mastra/fastembed` 1.3.0.

One provider note: an Anthropic-compatible endpoint that emits a `thinking` block with no signature needs `providerOptions.anthropic.thinking.type = 'disabled'`. Ours does. It is also 3.6× faster than the alternative we measured (6.2s vs 22.6s).